

Capstone Project

*06 July 2026
AI Honeywell Internship*

Abhinav Khandekar
Collage: Government Polytechnic Nagpur
Contact : ask15gpn@gmail.com
Ph: 9699610903

Project Title: Automobile Analysis and Predictions Report.

Objectives:

The objective of this project is to analyze automobile data, clean and preprocess the dataset, visualize important information, and build machine learning models to predict car-related values. The project also compares different machine learning algorithms to find the best-performing model.

Problem Statement:

Choosing the right car characteristics and predicting vehicle-related information manually is difficult. This project uses machine learning algorithms to analyze automobile data and make predictions based on different car features.

Dataset Description:

The dataset contains information about different car brands and their specifications. It includes features such as car make, fuel type, body style, drive wheels, wheel base, and normalized losses. These features are used for data analysis and prediction.

Loading dataset:

Dataset link: <https://www.kaggle.com/datasets/toramky/automobile-dataset>

Loading Dataset.

```
df = pd.read_csv(r"Automobile_data-selected-columns.csv") #Load file
df
```

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base
0	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6
1	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6
2	1	?	alfa-romero	gas	std	two	hatchback	rwd	front	94.5
3	2	164	audi	gas	std	four	sedan	fwd	front	99.8
4	2	164	audi	gas	std	four	sedan	4wd	front	99.4
...	...	...	...	...	...	...	...	...	...	...
200	-1	95	volvo	gas	std	four	sedan	rwd	front	109.1
201	-1	95	volvo	gas	turbo	four	sedan	rwd	front	109.1
202	-1	95	volvo	gas	std	four	sedan	rwd	front	109.1
203	-1	95	volvo	diesel	turbo	four	sedan	rwd	front	109.1
204	-1	95	volvo	gas	turbo	four	sedan	rwd	front	109.1

205 rows x 10 columns

205 rows x 10 columns

Data Preprocessing Steps:

- Removed unnecessary columns such as **Unnamed: 0**.

Removing un-named column.

```
df = df.drop("Unnamed: 0",axis = 1)
df.columns
```

[]

- Handled missing values.
- Converted categorical values into numerical values using encoding.
- Selected important features for model training.

- Capitalize rows and remove unnecessary spaces.

```
#Capitalize and keeping unique values.
print(df["make"].unique())
df['make']=(df['make'].str.strip().str.capitalize())
print(df['aspiration'].unique())
df['aspiration']=(df['aspiration'].str.strip().str.capitalize())
print(df['body-style'].unique())
df['body-style']=(df['body-style'].str.strip().str.capitalize())
print(df['drive-wheels'].unique())
df['drive-wheels']=(df['drive-wheels'].str.strip().str.capitalize())
print(df['engine-location'].unique())
df['engine-location']=(df['engine-location'].str.strip().str.capitalize())
print(df['fuel-type'].unique())
df['fuel-type']=(df['fuel-type'].str.strip().str.capitalize())
print(df['num-of-doors'].unique())
df['num-of-doors']=(df['num-of-doors'].str.strip().str.capitalize())
```

```
#Replacing nulls in 'num-of-doors'
df['num-of-doors'].isnull().sum() #found nulls.
df['num-of-doors'] = df['num-of-doors'].fillna(df['num-of-doors'].mode()[0])
```

Removing Duplicates.

```
df = df.drop_duplicates()
```

Also removed duplicates here.

- Also removed nulls:

Cleaning and Pre-processing data.

```
#Cleaning ? values.
df['normalized-losses'] = df['normalized-losses'].replace("?", df['normalized-losses'].mean())
df['num-of-doors'] = df['num-of-doors'].replace("?", "Two")
```

```
#Checking and Cleaning null values.
print(df.isnull().sum())
#Confirms if null or not.
```

- Last step to see if all columns have valid values (see comments for parameters):

Checking if numeric columns have invalid values.

```
print(df["symboling"].unique())
print(df["symboling"].describe()) #After checking there are no invalid values(between -3 to 3).
print(df["wheel-base"].unique())
print(df["wheel-base"].describe()) #After checking there are no invalid values(between 80-120).
```

- Check statistics of data before encoding:

Getting dataset statistical information.

```
#Get information of data and understand data.
df.describe()
df.info()
df.columns
```

[4] ✓ 1.7s

```
... <class 'pandas.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              205 non-null   int64
1   normalized-losses      205 non-null   str
2   make                   205 non-null   str
3   fuel-type              205 non-null   str
4   aspiration              205 non-null   str
5   num-of-doors            205 non-null   str
6   body-style              205 non-null   str
7   drive-wheels            205 non-null   str
8   engine-location         205 non-null   str
9   wheel-base             205 non-null   float64
dtypes: float64(1), int64(1), str(8)
memory usage: 16.1 KB

... Index(['symboling', 'normalized-losses', 'make', 'fuel-type', 'aspiration',
        'num-of-doors', 'body-style', 'drive-wheels', 'engine-location',
        'wheel-base'],
        dtype='str')
```

Encoding data: It's done mainly to make categorical columns numeric and assign each value a digit starting from 0.

Encoding data.

Using new data file .csv which is cleaned.

```
df = pd.read_csv(r"cleaned.csv")
```

[]

```
le = LabelEncoder()
cat_cols = df.select_dtypes(include=['object'])
print(cat_cols)
for col in cat_cols:
    df[col] = le.fit_transform(df[col])#Using new data file .csv which is cleaned.
```

[]

Encoding helps in creating co-relation matrix and useful in model creation and evaluation.

Exploratory Data Analysis (EDA):

- We created bar-graphs, histograms, pie charts, correlation matrix, heatmaps and scatter plots to visualize our data through varied insights. (Check commented code in screenshots for insights).
- These visualizations helped identify patterns and understand the data before training the model.
- First we check the co-relation between columns using corr() and heatmap:

Find data correlation using .corr() and heatmap.

```
#Finding Co-relation matrix.
corr = df.corr()
corr
```

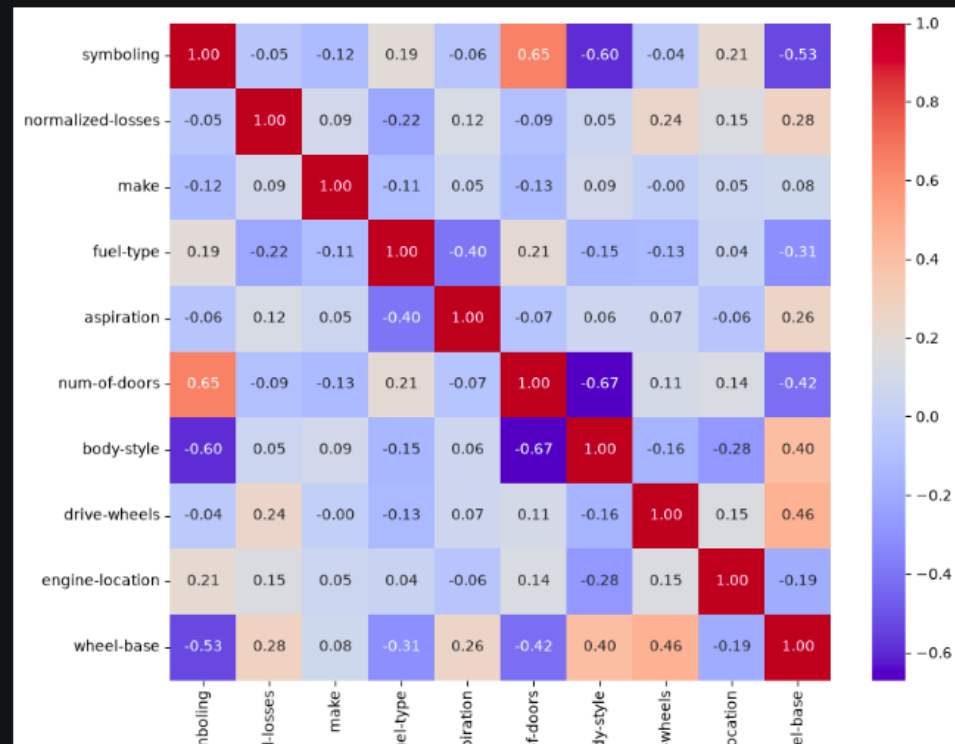
[147] ✓ 0.0s

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base
symboling	1.000000	-0.050579	-0.118794	0.194311	-0.059866	0.645492	-0.596135	-0.041671	0.212471	-0.531954
normalized-losses	-0.050579	1.000000	0.089937	-0.215545	0.123036	-0.090388	0.051574	0.244877	0.150216	0.278059
make	-0.118794	0.089937	1.000000	-0.113191	0.054265	-0.127416	0.089494	-0.004317	0.054608	0.078505
fuel-type	0.194311	-0.215545	-0.113191	1.000000	-0.401397	0.207665	-0.147853	-0.132257	0.040070	-0.308346
aspiration	-0.059866	0.123036	0.054265	-0.401397	1.000000	-0.066737	0.063028	0.066465	-0.057191	0.257611
num-of-doors	0.645492	-0.090388	-0.127416	0.207665	-0.066737	1.000000	-0.670198	0.112398	0.136578	-0.416331
body-style	-0.596135	0.051574	0.089494	-0.147853	0.063028	-0.670198	1.000000	-0.155745	-0.277009	0.401362
drive-wheels	-0.041671	0.244877	-0.004317	-0.132257	0.066465	0.112398	-0.155745	1.000000	0.147865	0.459745
engine-location	0.212471	0.150216	0.054608	0.040070	-0.057191	0.136578	-0.277009	0.147865	1.000000	-0.187790
wheel-base	-0.531954	0.278059	0.078505	-0.308346	0.257611	-0.416331	0.401362	0.459745	-0.187790	1.000000

Generate Code Markdown

```
plt.figure(figsize = (10,8))
sns.heatmap(corr, annot = True,cmap = "coolwarm", fmt = ".2f")
plt.show()
```

[148] ✓ 0.4s



Live Share

- **Now we check some other insights:**

1. Distribution of Car Brands ,using Bar graph:

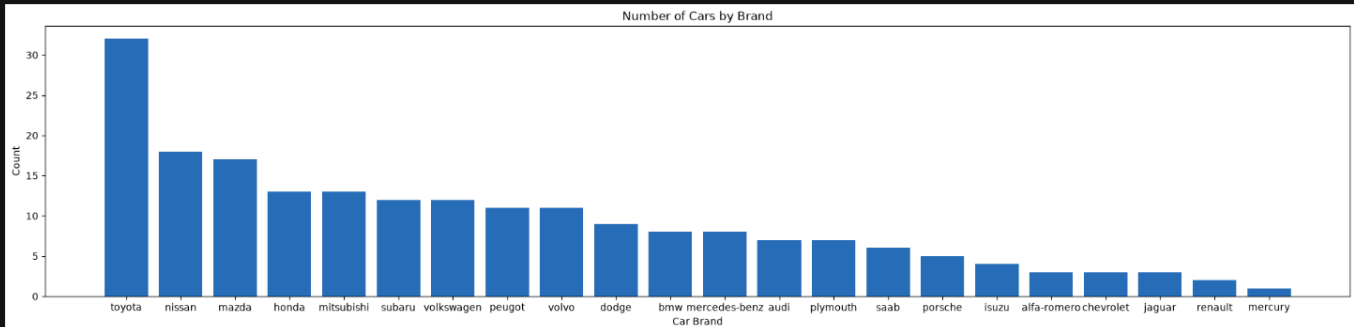
Insights: Toyota, Nissan, and Mazda have the highest number of cars in the dataset.
Luxury brands such as Jaguar and Porsche appear much less frequently.

1. Distribution of Car Brands ,using Bar graph:

```
brand_counts = df["make"].value_counts()
plt.figure(figsize=(24,5))
plt.bar(brand_counts.index, brand_counts.values)
plt.title("Number of Cars by Brand")
plt.xlabel("Car Brand")
plt.ylabel("Count")
plt.show()
```

[131] ✓ 0.2s

Python



2. Fuel Type Distribution, using pie chart:

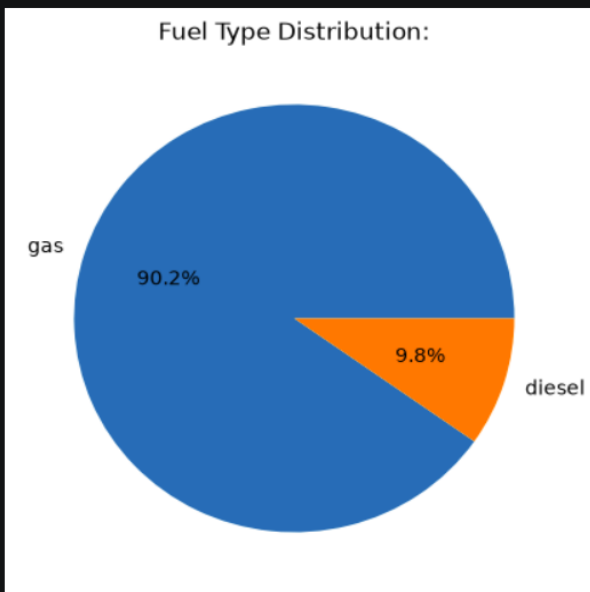
Insights: #Most cars use gas as their fuel type.

Diesel vehicles make up a much smaller portion of the dataset.

2. Fuel Type Distribution, using pie chart:

```
fuel_counts = df["fuel-type"].value_counts()
plt.figure(figsize=(5,5))
plt.title("Fuel Type Distribution:")
plt.pie(fuel_counts, labels = fuel_counts.index, autopct = "%1.1f%%")
plt.show()
```

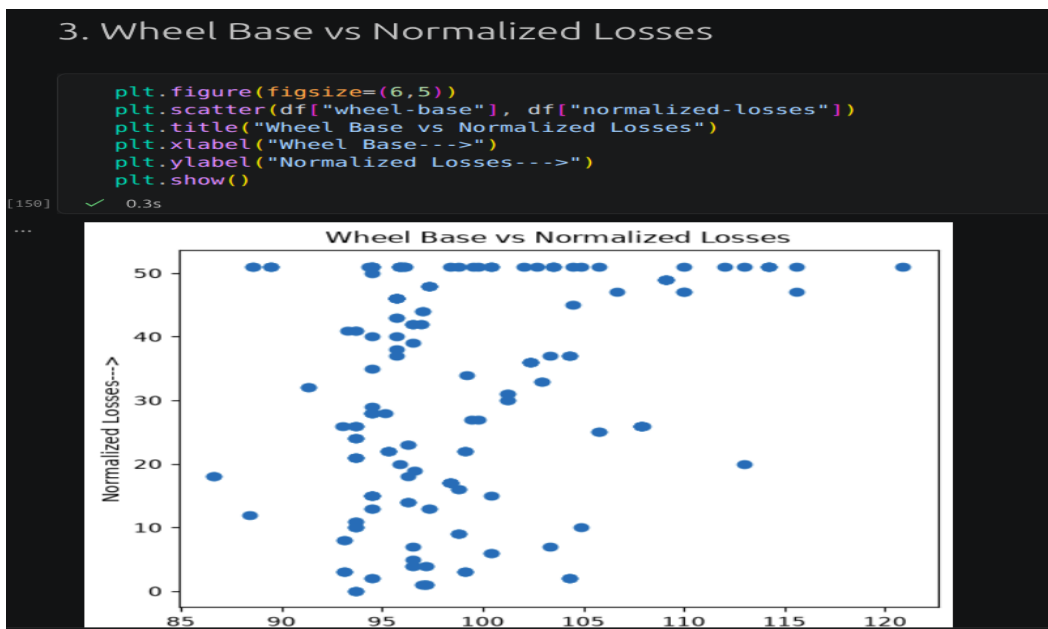
[139] ✓ 0.0s



3. Wheel Base vs Normalized Losses:

Insights : There is no strong linear relationship between wheel base and normalized losses.

Other vehicle characteristics are likely more influential.



4. Body Style by Drive Wheels:

Insights: Different body styles are associated with different drive-wheel configurations.



Feature Eng

The proj
It is mo

ation.
style, and drive-

wheels were encoded into numerical values so they could be used by machine learning algorithms.

Feature Engineering snapshot was placed in Data pre-processing section.

Machine Learning Algorithms Used (Other logic in images in next section):

1. Linear Regression:

Linear Regression is a supervised machine learning algorithm used for predicting **continuous numerical values**. It learns the relationship between independent variables and a target variable by fitting the best possible straight line through the data. In this project, Linear Regression was implemented **independently** to predict the **normalized-losses** value of an automobile. After training the model, it was evaluated using regression metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), and R² Score. The model produced an **MAE of 0.20**, **MSE of 0.20**, and an **R² Score of -0.25**. The negative R² score indicates that the linear relationship between the selected features and the target variable is weak, suggesting that a different regression algorithm or additional feature engineering could improve the prediction performance. **Below is logic given:**

1. Import the Linear Regression algorithm.
2. Select the independent variables (x) and target variable (normalized-losses).
3. Split the dataset into training and testing sets.
4. Create the Linear Regression model.
5. Train the model using the training dataset.
6. Predict the values for the testing dataset.
7. Evaluate the model using MAE, MSE, and R² Score.
8. Analyze the results to understand the model's prediction performance.

2. Logistics Regression:

Logistic Regression is a supervised machine learning algorithm used for **binary classification** problems. It estimates the probability of an input belonging to a particular class using the logistic (sigmoid) function. In this project, Logistic Regression was implemented **independently** to classify the **fuel type** of automobiles. The model was trained using the processed dataset and evaluated on the testing data. It achieved an **accuracy score of 80%**, indicating that it correctly classified most of the fuel types in the dataset. Although its performance was good, it was slightly lower than the KNN model. **Below logic is given:**

1. Import the Logistic Regression algorithm.
2. Select the input features (x) and target variable (fuel-type).
3. Split the dataset into training and testing sets.
4. Create the Logistic Regression model.
5. Train the model using the training data.
6. Predict the fuel type for the testing data.
7. Calculate the Accuracy Score.
8. Compare the result with other classification models.

From here we used a for-loop logic that can be seen in next section in images to represent other models workflow and accuracy score as these are solely based on Classification.

3. K-Nearest Neighbor:

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification. It predicts the class of a new data point by finding the K nearest training samples and assigning the class that appears most frequently among those neighbors. In this project, KNN was trained using the automobile dataset to predict the fuel type of a car based on its features. After comparing all classification models, KNN achieved the highest accuracy of 83.33%, making it the best-performing model. Therefore, KNN was selected to make predictions for new car data entered by the user. **Below is logic given:**

1. Import the KNeighborsClassifier.

2. Create the KNN model with `n_neighbors = 3`.
3. Train the model using the training dataset.
4. Predict the fuel type for the testing dataset.
5. Calculate the accuracy score.
6. Since it achieved the highest accuracy, use the trained KNN model to predict the fuel type of a new car.

4. Decision Tree:

Decision Tree is a supervised machine learning algorithm that makes predictions by splitting the dataset into different branches based on feature values. Each split helps the model reach a final decision. In this project, the Decision Tree classifier was trained along with the other classification models using a for loop. The model achieved an **accuracy of 80%**, which was slightly lower than KNN. Although its performance was good, KNN provided better prediction accuracy on this dataset.

5. Random Forest:

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction performance and reduce overfitting. Each tree makes its own prediction, and the final output is selected using majority voting. In this project, Random Forest was also trained using the same dataset inside the for loop. The model achieved an **accuracy of 80%**. While it produced reliable predictions, its accuracy was slightly lower than the KNN model.

Common logic:

- Import all three classification algorithms.
- Store the models in a Python dictionary.
- Create an empty dictionary to store the trained models.
- Use a **for loop** to train each model one by one.
- Predict the testing dataset for each model.
- Calculate and display the accuracy score of every model.
- Compare the accuracy values.
- Select the model with the highest accuracy (KNN).
- Store the trained KNN model and use it to predict the fuel type of a new car.

New Car logic:

After comparing the accuracy of all three classification models, **KNN** was selected because it achieved the highest accuracy (**83.33%**). A new car was then created by providing its feature values in the same format as the training dataset. The trained KNN model predicted the fuel type of this new car. Based on the prediction result, the model identified the car as a **Gas** vehicle. **Below logic is given:**

1. Select the best-performing model based on the highest accuracy score.
2. Create a new car data point by entering feature values in the same order as the training data.
3. Use the trained KNN model stored in the dictionary to predict the fuel type.
4. Store the prediction result in a variable.
5. Check the predicted class using an if-else statement.
6. Display the final prediction as **Gas** or **Diesel** based on the predicted value.

Model Evaluation:

First we see some results or cleaned data which our model is going to be trained on.

Capitalized data:

```

<StringArray>
[ 'alfa-romero',      'audi',      'bmw',      'chevrolet',
  'dodge',           'honda',     'isuzu',    'jaguar',
  'mazda', 'mercedes-benz', 'mercury',  'mitsubishi',
  'nissan',           'peugot',   'plymouth', 'porsche',
  'renault',          'saab',     'subaru',   'toyota',
  'volkswagen',      'volvo']
Length: 22, dtype: str
<StringArray>
['std', 'turbo']
Length: 2, dtype: str
<StringArray>
['convertible', 'hatchback', 'sedan', 'wagon', 'hardtop']
Length: 5, dtype: str
<StringArray>
['rwd', 'fwd', '4wd']
Length: 3, dtype: str
<StringArray>
['front', 'rear']
Length: 2, dtype: str
<StringArray>
['gas', 'diesel']
Length: 2, dtype: str
<StringArray>
['two', 'four', 'Two']
Length: 3, dtype: str

```

Null Removed results:



Removing nulls:

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base
0	3	df['normalized-losses'].mean()	Alfa-romero	Gas	Std	Two	Convertible	Rwd	Front	88.6
2	1	df['normalized-losses'].mean()	Alfa-romero	Gas	Std	Two	Hatchback	Rwd	Front	94.5
3	2	164	Audi	Gas	Std	Four	Sedan	Fwd	Front	99.8
4	2	164	Audi	Gas	Std	Four	Sedan	4wd	Front	99.4
5	2	df['normalized-losses'].mean()	Audi	Gas	Std	Two	Sedan	Fwd	Front	99.8
...	...	...	...	...	...	...	...	...	...	...
198	-2	103	Volvo	Gas	Turbo	Four	Sedan	Rwd	Front	104.3
199	-1	74	Volvo	Gas	Turbo	Four	Wagon	Rwd	Front	104.3
200	-1	95	Volvo	Gas	Std	Four	Sedan	Rwd	Front	109.1
201	-1	95	Volvo	Gas	Turbo	Four	Sedan	Rwd	Front	109.1
203	-1	95	Volvo	Diesel	Turbo	Four	Sedan	Rwd	Front	109.1

147 rows x 10 columns

Removing invalid “?” symbols so model could read and train on data accurately:

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base
0	3	df['normalized-losses'].mean()	alfa-romero	gas	std	two	convertible	rwd	front	88.6
1	3	df['normalized-losses'].mean()	alfa-romero	gas	std	two	convertible	rwd	front	88.6
2	1	df['normalized-losses'].mean()	alfa-romero	gas	std	two	hatchback	rwd	front	94.5
3	2	164	audi	gas	std	four	sedan	fwd	front	99.8
4	2	164	audi	gas	std	four	sedan	4wd	front	99.4
...	...	...	...	...	...	...	...	...	...	...
200	-1	95	volvo	gas	std	four	sedan	rwd	front	109.1
201	-1	95	volvo	gas	turbo	four	sedan	rwd	front	109.1
202	-1	95	volvo	gas	std	four	sedan	rwd	front	109.1
203	-1	95	volvo	diesel	turbo	four	sedan	rwd	front	109.1
204	-1	95	volvo	gas	turbo	four	sedan	rwd	front	109.1

205 rows x 10 columns

Model Training:

Now we train the models using:

Using same every model is trained.

```
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size = 0.2,random_state = 42) #For training & testing.
```

80% data is what the model is trained on and left 20% is that what we will test on model.

- The training data was then used to train the machine learning models, allowing it to learn the relationship between the input features x with dropped columns with highest co-relation and the target variable y for which we took y_pred in every model to predict accuracy.
- During this process, the model identified patterns from the training data and learned how to make predictions. These patterns are our data trends which it is trained on.
- After the training was completed, the trained model was tested on unseen data to evaluate its performance and prediction accuracy.

Model Evaluation:

1. Linear Regression:

The performance of the Linear Regression model was evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), and R² Score.

- Mean Absolute Error (MAE): 0.20
- Mean Squared Error (MSE): 0.20
- R² Score: -0.25

The MAE and MSE values indicate the average prediction error of the model, while the R² Score measures how well the model explains the variation in the target variable. In this project, the R² Score was -0.25, indicating that the Linear Regression model did not fit the dataset well. This suggests that the relationship between the selected features and normalized-losses is not strongly linear, and the prediction performance could be improved by using additional features or more advanced regression algorithms in future work.

```
print("Mean absolute error:",mean_absolute_error(y_test,y_pred))
print("Mean squared error:",mean_squared_error(y_test,y_pred))
print("R2 score:",r2_score(y_test,y_pred))
```

[68] ✓ 0.0s

... Mean absolute error: 0.2
Mean squared error: 0.2
R2 score: -0.25

2. Logistics Regression:

The performance of the Logistic Regression model was evaluated using the **Accuracy Score**, which measures the percentage of correctly classified records. In this project, the model achieved an **accuracy of 80%**, meaning it correctly predicted the fuel type for most of the testing data. The result indicates that Logistic Regression performed well on the dataset and was able to classify the fuel type effectively. However, when compared with the other classification models, **K-Nearest Neighbors (KNN)** achieved a higher accuracy of **83.33%**, making it the best-performing model for this dataset. Therefore, KNN was selected as the final model for predicting the fuel type of new automobile data.

```
#Variable to store value to predict.
y_pred = model.predict(x_test)
```

[]

```
acc = accuracy_score(y_test,y_pred)
print("Accuracy score",acc)
```

[69] ✓ 0.0s

... Accuracy score 0.8

Accuracy score: 80% (0.8)

3. Combined result of K-Nearest Neighbor, Decision Tree, Random Forest:

The dataset was divided into training and testing sets using an 80:20 ratio. To compare the performance of different classification algorithms, K-Nearest Neighbors (KNN), Decision Tree, and Random Forest were stored in a dictionary and trained using a for loop. Each model learned patterns from the training data and was then used to make predictions on the testing dataset. This approach made it easy to train, test, and compare multiple models using the same dataset and evaluation process.

```
models = { "KNN": KNeighborsClassifier(n_neighbors = 3), "Decision Tree": DecisionTreeClassifier(random_state = 42),
           "Random Forest": RandomForestClassifier(random_state = 42)}
trained_models = {}

for name, model in models.items():
    model.fit(x_train, y_train)
    trained_models[name] = model
    y_pred = model.predict(x_test)
    accuracy = accuracy_score(y_test, y_pred)
    print(f"\n{name}")
    print(f"Accuracy: {accuracy:.4f}")

[70] ✓ 0.1s

...

KNN
Accuracy: 0.8333

Decision Tree
Accuracy: 0.8000

Random Forest
Accuracy: 0.8000
```

As we can see from all three's accuracy score,
Accuracy score of KNN is highest: 0.833 or 83% which is highest among all models.

Metrics and Results:

As from the screenshots we can see all the 'Classifier' based models we only used below given metric:

Accuracy Score:

Accuracy Score is an evaluation metric used to measure the performance of classification models. It calculates the percentage of predictions that were classified correctly out of the total number of predictions made. A higher accuracy score indicates better model performance. In this project, the accuracy score was used to compare the performance of K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and Logistic Regression models and identify the most suitable algorithm for predicting the fuel type of automobiles.

All results are available in screenshots in above section.

Regression Evaluation Metrics

Mean Absolute Error (MAE) : MAE measures the average difference between the actual and predicted values. A lower MAE indicates better model accuracy.

Mean Squared Error (MSE) : MSE calculates the average of the squared differences between actual and predicted values. Lower MSE values indicate better model performance, while larger errors are penalized more heavily.

R² Score: The R² Score measures how well the regression model explains the variation in the target variable. A value closer to **1** indicates a better fit, while a value close to **0** or negative indicates poor model performance.

Conclusion:

This project successfully demonstrated the complete process of applying machine learning techniques to an automobile dataset. The dataset was cleaned and preprocessed by removing unnecessary columns, handling missing values, and encoding categorical features into numerical values. Exploratory Data Analysis (EDA) was performed using different visualizations such as bar charts, pie charts, histograms, scatter plots, and a correlation heatmap, which helped in understanding the characteristics and distribution of the data.

Several machine learning algorithms, including K-Nearest Neighbors (KNN), Decision Tree, Random Forest, Logistic Regression, and Linear Regression, were implemented and evaluated. The models were trained using the processed dataset and tested on unseen data to measure their performance. Appropriate evaluation metrics were used to compare the models, and predictions were successfully generated for new automobile records.

We got best results from K-Nearest Neighbors (KNN),.

Future Scope and Improvements:

- I wanted to develop a simple frontend interface so users can easily enter car details and get predictions without using the notebook.
- Integrate the project with a database to store automobile data and prediction results for future use.
- Improve the accuracy of some machine learning models by tuning their parameters and selecting better features.
- Reduce overfitting in models where training accuracy is much higher than testing accuracy.
- Improve the code by making it more modular, readable, and easier to maintain.
- Add more meaningful data visualizations and insights to better understand the automobile dataset and support decision-making.